SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – Scenario-Based Requirement Analysis

Course Code:	CSA1007	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	RISHIK JK	**Reg. No.:**	192511052
Date:	29/08/2026	**Duration:**	60 minutes

Code of Conduct

*I **RISHIK JK (192511052)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **RISHIK JK**

CO4-AT2-CI/CD Pipeline Design Exercise

NAME: JK RISHIK **SLOT:** D

REGNO :192511052 **COURSE CODE:** CSA1007

DATE:29/08/2026

PROBLEM STATEMENT:Smart Renewable Battery Recycling Platform

1. Introduction

The Smart Renewable Battery Recycling Platform is developed to manage battery inventory, recycling operations, hazardous conditions, logistics, sustainability reports, and operator notifications. To develop and maintain the system efficiently, a CI/CD pipeline is designed to automate code integration, testing, quality checking, packaging, and deployment.

CI/CD helps the development team detect errors early, reduce manual work, and deliver software updates in a faster and more reliable way.

2. Project Requirements and CI/CD Needs

The system contains different modules such as inventory management, recycling operations, hazard detection, logistics tracking, reporting, and notifications. Since these modules may be developed and updated frequently, an automated pipeline is required.

The main CI/CD requirements are:

- Automatically detect new code changes.
- Build the application whenever code is updated.
- Run automated tests before deployment.
- Check code quality and security.

- Package the application using Docker.
- Deploy the application to testing and production environments.
- Send notifications if a build or deployment fails.
- Support rollback when a new version causes errors.

3. CI/CD Pipeline Workflow

The proposed CI/CD pipeline contains the following stages:

3.1 Source Code Management

The developers store and manage the project code using **GitHub**. Whenever a developer commits and pushes code to the repository, the CI/CD pipeline starts automatically.

3.2 Build Stage

The application source code is prepared and required dependencies are installed. The build stage checks whether the application can be created successfully without errors.

3.3 Automated Testing

Automated tests are executed to verify important system functions such as login, inventory management, recycling operations, hazard detection, logistics, and report generation.

3.4 Code Quality Analysis

The code is checked for coding errors, duplicate code, security issues, and maintainability problems before packaging.

3.5 Packaging

After successful testing, the application is packaged into a **Docker image**. This ensures that the application runs in the same environment during development, testing, and deployment.

3.6 Deployment

The Docker image is first deployed to the staging environment for final checking. After successful validation, the same application is deployed to the production environment.

4. Tools and Technologies

Git and GitHub

Git is used for version control, and GitHub is used for storing the project source code. It allows multiple developers to work on the same project and maintain different versions of the application.

GitHub Actions

GitHub Actions is used to automate the CI/CD workflow. It can automatically start the build, test, and deployment process whenever new code is pushed to the repository.

Python and Flask

Python with Flask can be used to develop the backend application. Flask is simple and suitable for developing web-based applications.

MySQL

MySQL is used to store user details, battery inventory, recycling records, logistics information, and sustainability data.

Docker

Docker is used to package the application and its dependencies into a container. It ensures that the application behaves consistently in different environments.

PyTest

PyTest is used for automated testing of the Python application. It helps verify important application functions before deployment.

SonarQube

SonarQube can be used to check code quality, bugs, code duplication, and maintainability issues.

5. Automated Testing Strategy

Automated testing is an important part of the CI/CD pipeline because faulty code should not be deployed.

The following testing strategies are included:

Unit Testing

Unit testing checks individual functions and modules separately.

Examples:

- User login validation.
- Battery quantity validation.
- Hazard detection function.
- Report calculation.

Integration Testing

Integration testing verifies whether different modules work correctly together.

For example, when a hazardous battery condition is detected, the hazard detection module should communicate with the notification module and send an alert.

Functional Testing

Functional testing checks whether the major features of the platform work according to requirements.

It includes:

- Battery inventory management.
- Recycling task assignment.
- Logistics tracking.
- Sustainability report generation.

Security Testing

Security checks are performed to verify user authentication, role-based access, and unauthorized access attempts.

Regression Testing

Regression testing ensures that new changes do not affect existing features that were already working correctly.

If any automated test fails, the pipeline should stop and the new version should not be deployed.

6. Deployment Strategy

The proposed deployment process uses three main environments.

6.1 Development Environment

Developers create and test new features in the development environment. This environment is mainly used during coding.

6.2 Testing / Staging Environment

After the application passes automated tests, it is deployed to the staging environment. This environment is similar to the real production system and is used for final verification.

6.3 Production Environment

After successful testing in the staging environment, the approved application is deployed to production. Production is the live environment used by actual users.

The same Docker image should be used for staging and production to maintain consistency.

7. Security and Quality Checks

Security and quality checks are included in the pipeline before deployment.

The important checks include:

- Secure storage of passwords and API keys.
- Role-based access control.
- Code quality analysis.
- Dependency security checking.
- Prevention of unauthorized deployment.
- Automated test verification.
- Database backup before major releases.

Sensitive information such as passwords should not be stored directly in the source code repository. Environment variables or secure secrets should be used instead.

8. Notification Mechanism

Notifications help the development team quickly identify pipeline problems.

The pipeline can send notifications when:

- Build is successful.
- Build fails.
- Automated tests fail.
- Deployment is completed.
- Production deployment fails.

Notifications can be provided through email or development communication platforms.

9. Rollback Mechanism

Rollback is used when a newly deployed version causes errors in the production system.

Each Docker image can be stored with a separate version tag. If the latest version fails, the previous stable Docker image can be deployed again.

For example:

Version 1.0 → Stable

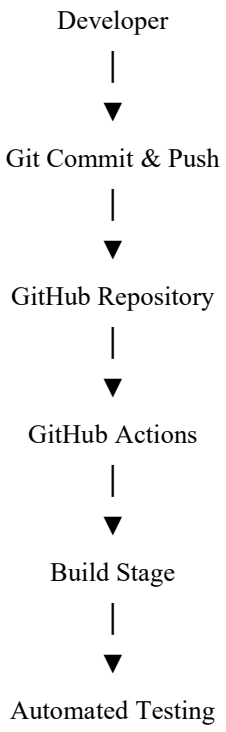
Version 1.1 → New Deployment

If Version 1.1 fails, the system can return to Version 1.0.

This reduces downtime and prevents serious problems for users.

10. CI/CD Pipeline Diagram

Draw this diagram using Shapes in Microsoft Word.



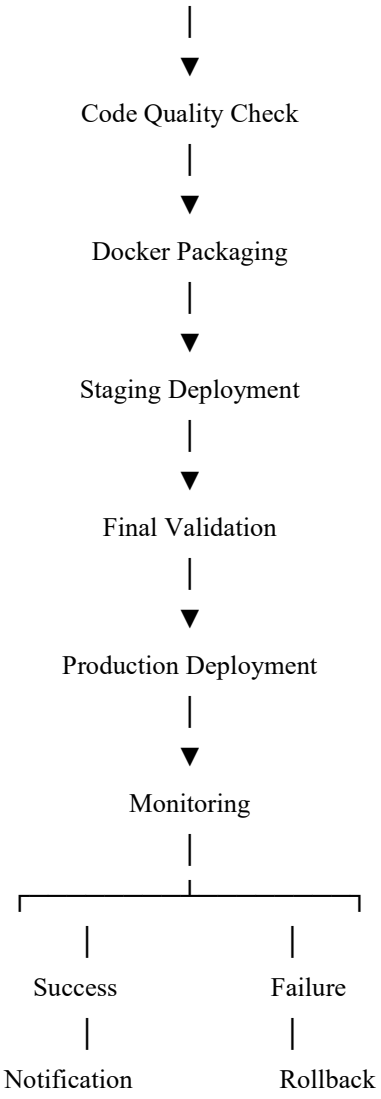


Figure 1: CI/CD Pipeline for Smart Renewable Battery Recycling Platform

11. CI/CD Pipeline Workflow Explanation

The workflow begins when a developer makes changes to the project and pushes the updated code to the GitHub repository.

GitHub Actions automatically starts the CI/CD pipeline. The application is built, dependencies are installed, and automated tests are executed. If the tests are successful, code-quality and security checks are performed.

After all checks are completed successfully, Docker packages the application as an image. The image is first deployed to the staging environment for final testing. If the staging version works correctly, it is deployed to the production environment.

If any stage fails, the pipeline stops the deployment and sends a notification to the development team. If a problem occurs after production deployment, the previous stable version is restored using the rollback mechanism.

12. Advantages of the Proposed CI/CD Pipeline

The proposed CI/CD pipeline provides several benefits:

- Reduces manual development and deployment work.
- Detects errors at an early stage.
- Improves software quality.
- Provides consistent deployment using Docker.
- Makes new software updates faster.
- Prevents faulty code from reaching production.
- Improves collaboration between team members.
- Supports quick rollback during failures.

13. Justification of the CI/CD Design

The selected pipeline is suitable for the Smart Renewable Battery Recycling Platform because the system contains several modules that may require frequent changes and improvements.

GitHub provides simple source code management, while GitHub Actions automates the workflow. PyTest helps verify application functionality, SonarQube improves code quality, and Docker provides consistent application packaging.

Using separate development, staging, and production environments reduces deployment risk. Security checks, notifications, and rollback mechanisms further improve the reliability of the system.

The selected approach is also simple enough to maintain while supporting future system growth and additional modules.

14. Conclusion

The CI/CD pipeline designed for the Smart Renewable Battery Recycling Platform automates code integration, building, testing, quality analysis, packaging, and deployment. The pipeline uses suitable tools such as GitHub, GitHub Actions, PyTest, SonarQube, Docker, and MySQL.

The use of automated testing, security checks, staging deployment, notifications, and rollback mechanisms helps maintain software quality and reliability. Overall, the proposed CI/CD pipeline provides a structured and efficient method for developing and deploying the Smart Renewable Battery Recycling Platform.